

데이터베이스 구동 및 CRUD

Docker Compose로 PostgreSQL 만 실행하고 FastAPI는 로컬에서 venv로 실행하는 예제 입니다.

1. 최종 목표

구성은 아래와 같습니다.

- Docker Compose
 - PostgreSQL만 실행
- Windows 로컬 Python venv
 - FAST API 실행
 - SQLAlchemy ORM 사용
- 기능
 - User Table 생성
 - 사용자 생성 (User)
 - 사용자 목록 조회 (Read List)
 - 사용자 단건 조회 (Read One)
 - 사용자 수정 (Update)
 - 사용자 삭제 (Delete)
- 문서
 - Swagger UI 자동 생성
 - ReDoc 자동 생성

2. 프로젝트 폴더 구조

```
C:\projects\fastapi-postgres-local
|
├─ app/
│  ├── **init**.py
│  ├── main.py
│  ├── database.py
│  ├── models.py
│  ├── schemas.py
│  └─ crud.py
|
├─ requirements.txt
├─ docker-compose.yml
└─ .env
```

3. 파일 만들기

3-1. requirements.txt

```
fastapi
uvicorn
sqlalchemy
psycopg2-binary
pydantic
email-validator
python-dotenv
```

설명:

```
fastapi: 웹 API 프레임워크
uvicorn: FastAPI 실행 서버
sqlalchemy: ORM
psycopg2-binary: PostgreSQL 드라이버
pydantic: 요청/응답 데이터 검증
email-validator: EmailStr 검증용
python-dotenv: .env 파일 환경변수 로드용
```

3-2. .env

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=1234
POSTGRES_DB=fastapi_db
DATABASE_URL=postgresql://postgres:1234@localhost:5432/fastapi_db
```

설명:

FastAPI는 로컬 Windows에서 실행
PostgreSQL은 Docker 컨테이너에서 실행
컨테이너 포트 5432를 로컬 5432에 연결하므로
FastAPI에서는 DB 주소를 localhost:5432로 접속합니다

3-3. docker-compose.xml

```
services:
  db:
    image: postgres:16
    container_name: fastapi-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 1234
      POSTGRES_DB: fastapi_db
    ports:
```

```

- "5432:5432"
volumes:
- postgres_data:/var/lib/postgresql/data

volumes:
postgres_data:

```

설명:

```

db: PostgreSQL 서비스 이름
image: postgres:16: PostgreSQL 16 이미지 사용
ports: 로컬 5432 ↔ 컨테이너 5432 연결
volumes: DB 데이터 영속 저장

```

3-4. app/database.py

```

import os
# os 모듈은 운영체제의 환경변수(Environment Variable)를 읽을 때 사용합니다.
# 예: DATABASE_URL, POSTGRES_USER 같은 값을 코드 바깥에서 주입할 수 있습니다.

from dotenv import load_dotenv
# load_dotenv()는 .env 파일에 작성한 환경변수를 현재 파이썬 실행 환경에 로드합니다.

from sqlalchemy import create_engine
# create_engine():
# SQLAlchemy가 DB와 통신하기 위한 "엔진" 객체를 생성하는 함수입니다.
# 쉽게 말해, PostgreSQL과 연결하기 위한 핵심 통로를 만듭니다.

from sqlalchemy.orm import sessionmaker, declarative_base
# sessionmaker():
# DB 작업용 Session 객체를 생성하는 "세션 공장(factory)"입니다.
# SessionLocal()을 호출하면 실제 DB 세션 객체가 만들어집니다.
#
# declarative_base():
# ORM 모델 클래스들이 상속받는 기본 부모 클래스를 생성합니다.
# class User(Base): 처럼 사용합니다.

# .env 파일 로드
load_dotenv()

# os.getenv("환경변수명", 기본값)
# - 운영체제/환경에 해당 이름의 환경변수가 있으면 그 값을 사용
# - 없으면 두 번째 인자인 기본값 사용
#
# 즉, 여기서는 DATABASE_URL 환경변수가 존재하면 그 값을 사용하고
# 없으면 기본 PostgreSQL 접속 문자열을 사용합니다.
DATABASE_URL = os.getenv(

```

```

"DATABASE_URL",
"postgresql://postgres:1234@localhost:5432/fastapi_db"
)

# create_engine(...)
# - 실제 DB 연결용 엔진 생성
# - echo=True: SQLAlchemy가 내부적으로 실행하는 SQL문을 콘솔에 출력
#   학습과 디버깅에 매우 유용합니다.
engine = create_engine(DATABASE_URL, echo=True)

# sessionmaker(...)
# - DB 세션 생성 규칙을 정의하는 공장 함수
#
# autocommit=False
# - commit을 자동으로 하지 않음
# - 즉, db.commit()을 직접 호출해야 INSERT/UPDATE/DELETE가 실제 반영됨
#
# autoflush=False
# - 세션 변경사항을 자동 flush 하지 않음
# - 학습 단계에서는 commit/refresh 흐름을 이해하기 좋음
#
# bind=engine
# - 어떤 DB 엔진과 연결되는 세션인지 지정
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# declarative_base()
# - ORM 모델들의 부모 클래스 Base 생성
# - Base를 상속한 클래스들이 SQLAlchemy ORM 모델로 인식됨
Base = declarative_base()

```

3-5. app/models.py

SQLAlchemy 2.0 형식

```

from sqlalchemy import Integer, String
# Integer, String 은 DB 컬럼 타입입니다.

from sqlalchemy.orm import Mapped, mapped_column
# Mapped[T]:
# SQLAlchemy 2.0 스타일에서 ORM 필드를 타입 힌트와 함께 선언할 때 사용합니다.
#
# mapped_column(...):
# 기존 Column(...) 대신 사용하는 2.0 스타일 컬럼 선언 함수입니다.

```

```
from .database import Base
# Base는 ORM 모델들의 공통 부모 클래스입니다.

class User(Base):
    # User 클래스는 users 테이블과 연결되는 ORM 모델입니다.

    __tablename__ = "users"
    # 실제 DB에 생성될 테이블 이름

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        index=True
    )
    # id 컬럼
    # - Integer: 정수형
    # - primary_key=True: 기본키
    # - index=True: 인덱스 생성

    name: Mapped[str] = mapped_column(
        String(100),
        nullable=False
    )
    # name 컬럼
    # - String(100): 최대 100자
    # - nullable=False: 필수값

    email: Mapped[str] = mapped_column(
        String(200),
        unique=True,
        index=True,
        nullable=False
    )
    # email 컬럼
    # - String(200): 최대 200자
    # - unique=True: 중복 불가
    # - index=True: 검색 성능 향상
    # - nullable=False: 필수값

    age: Mapped[int | None] = mapped_column(
        Integer,
        nullable=True
    )
    # age 컬럼
    # - Integer: 정수형
    # - nullable=True: 선택값
    # - int | None: Python 타입 힌트상 None 허용
```

3-6. app/schemas.py

```
from pydantic import BaseModel, EmailStr
# BaseModel:
# FastAPI에서 요청/응답 데이터 검증에 사용하는 기본 클래스입니다.
#
# EmailStr:
# 이메일 형식인지 자동 검증해주는 타입입니다.

from typing import Optional
# Optional[T]:
# 값이 T이거나 None일 수 있음을 의미합니다.

class UserCreate(BaseModel):
    # 사용자 생성 요청용 스키마
    # POST /users 요청 본문(JSON)의 구조를 정의합니다.

    name: str
    # 사용자 이름
    # 필수 입력값이며 문자열이어야 합니다.

    email: EmailStr
    # 사용자 이메일
    # 필수 입력값이며 이메일 형식이어야 합니다.

    age: Optional[int] = None
    # 사용자 나이
    # 선택 입력값이며 입력하지 않으면 None 입니다.

class UserUpdate(BaseModel):
    # 사용자 수정 요청용 스키마
    # PUT /users/{user_id} 요청에서 사용합니다.
    # 부분 수정이 가능해야 하므로 모든 필드를 Optional 처리합니다.

    name: Optional[str] = None
    # 이름 수정값
    # 보내지 않으면 기존 값 유지

    email: Optional[EmailStr] = None
    # 이메일 수정값
    # 보내지 않으면 기존 값 유지
    # 보낸 경우 이메일 형식 검증 수행

    age: Optional[int] = None
    # 나이 수정값
    # 보내지 않으면 기존 값 유지

class UserResponse(BaseModel):
    # 응답용 스키마
    # DB에서 가져온 User ORM 객체를 어떤 형식으로 응답할지 정의합니다.
```

```

id: int
# 사용자 ID

name: str
# 사용자 이름

email: EmailStr
# 사용자 이메일

age: Optional[int] = None
# 사용자 나이 (없을 수 있음)

class Config:
    from_attributes = True
    # SQLAlchemy ORM 객체를 Pydantic 응답 모델로 변환할 수 있게 해줍니다.
    # 즉, dict가 아니라 ORM 객체(User)를 그대로 반환해도
    # FastAPI가 자동으로 UserResponse 형태로 변환합니다.

```

3-7. app/crud.py

```

from sqlalchemy.orm import Session
# Session:
# SQLAlchemy에서 DB 작업을 수행하는 세션 객체 타입입니다.

from . import models, schemas
# models: ORM 모델
# schemas: Pydantic 스키마

def create_user(db: Session, user: schemas.UserCreate):
    # 사용자 생성 함수
    # - db: DB 세션
    # - user: 사용자 생성 요청 데이터

    db_user = models.User(
        name=user.name,
        email=user.email,
        age=user.age
    )
    # Pydantic 스키마 값을 이용하여
    # 실제 DB에 저장할 ORM 객체 생성

    db.add(db_user)
    # db.add(...)
    # - 이 객체를 DB 저장 대상으로 세션에 등록합니다.
    # - 아직 실제 DB에 INSERT가 확정된 것은 아닙니다.
    # - Session이 이 객체를 추적하도록 등록하는 단계입니다.

    db.commit()
    # db.commit()
    # - 현재 세션의 변경사항을 실제 DB에 반영합니다.

```

```
# - create에서는 INSERT SQL이 실행됩니다.
# - commit 전까지는 트랜잭션 내부 변경 상태입니다.

db.refresh(db_user)
# db.refresh(...)
# - DB에 저장된 최신 상태를 다시 ORM 객체에 반영합니다.
# - 자동 생성된 id 값 등을 객체에 채워 넣습니다.

return db_user

def get_users(db: Session):
    # 전체 사용자 목록 조회
    return db.query(models.User).all()

def get_user(db: Session, user_id: int):
    # 특정 사용자 1명 조회
    # 없으면 None 반환
    return db.query(models.User).filter(models.User.id == user_id).first()

def update_user(db: Session, user_id: int, user_data: schemas.UserUpdate):
    # 사용자 수정 함수
    # - 대상 사용자를 찾고
    # - 전달된 값만 수정한 뒤
    # - commit/refresh 수행

    db_user = db.query(models.User).filter(models.User.id == user_id).first()

    if not db_user:
        return None

    if user_data.name is not None:
        db_user.name = user_data.name
        # name 값이 전달된 경우에만 수정

    if user_data.email is not None:
        db_user.email = user_data.email
        # email 값이 전달된 경우에만 수정

    if user_data.age is not None:
        db_user.age = user_data.age
        # age 값이 전달된 경우에만 수정

    db.commit()
    # commit을 호출해야 UPDATE 내용이 실제 DB에 반영됩니다.

    db.refresh(db_user)
    # DB에 반영된 최신 상태를 다시 ORM 객체에 동기화합니다.

    return db_user
```



```
def delete_user(db: Session, user_id: int):
    # 사용자 삭제 함수
    # - 대상 사용자를 찾고
    # - 존재하면 삭제 후 commit
    # - 없으면 None 반환

    db_user = db.query(models.User).filter(models.User.id == user_id).first()

    if not db_user:
        return None

    db.delete(db_user)
    # db.delete(...)
    # - ORM 객체를 삭제 대상으로 표시합니다.
    # - 아직 바로 DB에서 지워지는 것은 아니고
    #   commit 시점에 DELETE SQL이 실행됩니다.

    db.commit()
    # 삭제 내용을 실제 DB에 반영합니다.

    return db_user
```

3-8. app/main.py

```
from fastapi import FastAPI, Depends, HTTPException
# FastAPI:
# API 애플리케이션 객체 생성용 클래스
#
# Depends:
# 의존성 주입(Dependency Injection) 기능
# 여기서는 DB 세션을 각 API 함수에 자동 주입할 때 사용
#
# HTTPException:
# 에러 상황에서 HTTP 상태코드와 메시지를 반환할 때 사용

from sqlalchemy.orm import Session
# Session 타입 힌트용 import

from sqlalchemy.exc import IntegrityError
# IntegrityError:
# DB 제약조건 위반 시 발생하는 예외
# 예: unique=True인 email 중복 입력

from .database import SessionLocal, engine, Base
# SessionLocal: DB 세션 생성용
# engine: DB 연결 엔진
# Base: ORM 모델들의 부모 클래스

from . import schemas, crud
# schemas: 요청/응답 검증용 Pydantic 스키마
# crud: 실제 DB 처리 함수들
```

```

# Base.metadata.create_all(bind=engine)
# - Base를 상속한 모든 ORM 모델을 기준으로
#   DB에 테이블이 없으면 생성합니다.
# - 즉, 앱 시작 시 users 테이블이 자동 생성됩니다.
Base.metadata.create_all(bind=engine)

# FastAPI 앱 생성
app = FastAPI(
    title="FastAPI + PostgreSQL CRUD Example",
    description="Docker Compose로 PostgreSQL을 실행하고, FastAPI + SQLAlchemy로 User
CRUD를 수행하는 예제",
    version="1.0.0"
)
# title, description, version 값은 Swagger / ReDoc / OpenAPI 문서에 반영됩니다.

def get_db():
    # DB 세션 생성 및 종료를 담당하는 함수
    # Depends(get_db) 로 각 API에 주입됩니다.

    db = SessionLocal()
    # 실제 DB 세션 객체 생성

    try:
        yield db
        # API 함수에 db 세션 전달
    finally:
        db.close()
        # 요청 처리가 끝나면 세션 종료
        # 연결 자원 누수를 막기 위해 중요

@app.get("/", tags=["Root"])
def root():
    # 기본 접속 확인용 API
    return {"message": "FastAPI + PostgreSQL + SQLAlchemy CRUD"}

@app.post("/users", response_model=schemas.UserResponse, tags=["Users"])
def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    # 사용자 생성 API
    #
    # - user: 요청 본문(JSON)을 UserCreate 스키마로 검증한 값
    # - db: Depends(get_db)를 통해 주입받은 DB 세션
    #
    # response_model=UserResponse:
    # 반환값을 UserResponse 형식으로 변환해서 응답

    try:
        return crud.create_user(db, user)
    except IntegrityError:

```

```
db.rollback()
# rollback():
# commit 실패 시 현재 트랜잭션 상태를 되돌립니다.
# 중복 email 등 제약조건 위반 시 세션을 정상 상태로 복구하는 데 필요합니다.

raise HTTPException(status_code=400, detail="이미 존재하는 이메일입니다.")

@app.get("/users", response_model=list[schemas.UserResponse], tags=["Users"])
def read_users(db: Session = Depends(get_db)):
    # 전체 사용자 목록 조회 API
    return crud.get_users(db)

@app.get("/users/{user_id}", response_model=schemas.UserResponse, tags=["Users"])
def read_user(user_id: int, db: Session = Depends(get_db)):
    # 특정 사용자 조회 API
    # user_id는 URL 경로 변수(Path Parameter)입니다.

    db_user = crud.get_user(db, user_id)

    if not db_user:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")

    return db_user

@app.put("/users/{user_id}", response_model=schemas.UserResponse, tags=["Users"])
def update_user(user_id: int, user: schemas.UserUpdate, db: Session =
Depends(get_db)):
    # 사용자 수정 API

    try:
        db_user = crud.update_user(db, user_id, user)

        if not db_user:
            raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")

        return db_user

    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=400, detail="이미 존재하는 이메일입니다.")

@app.delete("/users/{user_id}", tags=["Users"])
def delete_user(user_id: int, db: Session = Depends(get_db)):
    # 사용자 삭제 API

    db_user = crud.delete_user(db, user_id)

    if not db_user:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")
```

```
return {"message": "사용자가 삭제되었습니다."}
```

3-9. app/init.py

```
# app 패키지를 인식시키기 위한 파일
```

4. 실행방법

4-1. 프로젝트 폴더로 이동

```
cd C:\Projects\fastapi-postgres-local
```

4-2. PostgreSQL 실행

```
docker compose up -d
```

확인

```
docker compose ps
```

정상 실행이면 fastapi-postgres 컨테이너가 떠 있어야 합니다.

4-3 Python 가상 환경 생성

```
python -m venv .venv
```

4-4 가상환경 활성화

CMD

```
venv\Scripts\activate
```

PowerShell

```
.\venv\Scripts\Activate.ps1
```

4-5. 패키지 설치

```
uvicorn app.main:app --reload
```

정상 실행되면 보통 아래와 비슷하게 나옵니다.

```
ubicorn running on http://127.0.0.1:8000
```

5. 접속 주소

5.1 기본 API 확인

브라우저

```
http://127.0.0.1:8000/
```

응답:

```
{  
  "message": "FastAPI + PostgreSQL + SQLAlchemy CRUD"  
}
```

5-2. Swagger UI

```
http://127.0.0.1:8000/docs
```

여기서 API를 직접 테스트 할 수 있습니다.

5-3. ReDoc 문서

```
http://127.0.0.1:8000/redoc
```

문서형 API 설명 페이지 입니다.

6. CRUD 테스트 방법

6-1. 사용자 생성

POST /users -> Try it out 입력:

```
{
  "name": "홍길동",
  "email": "hong@example.com",
  "age": 30
}
```

응답 예:

```
{
  "id": 1,
  "name": "홍길동",
  "email": "hong@example.com",
  "age": 30
}
```

6-2. 전체 사용자 조회

GET /users

```
[
  {
    "id": 1,
    "name": "홍길동",
    "email": "hong@example.com",
    "age": 30
  }
]
```

6-3. 사용자 단건 조회

GET /users/{user_id} 예: 1 응답 예:

```
{
  "id": 1,
  "name": "홍길동",
  "email": "hong@example.com",
  "age": 30
}
```

6-4. 사용자 수정

PUT /users/{user_id} 예: 1

입력:

```
{
  "name": "홍길순",
  "age": 31
}
```

응답 예:

```
{
  "id": 1,
  "name": "홍길순",
  "email": "hong@example.com",
  "age": 31
}
```

6-5. 사용자 삭제

PUT /users/{user_id} 예: 1 입력:

```
{
  "name": "홍길순",
  "age": 11
}
```

응답 예:

```
{
  "id": 1,
  "name": "홍길순",
  "email": "hong@example.com",
  "age": 31
}
```

7. PostgreSQL 내부 확인 방법

컨테이너 내부에서 pql 접속

```
docker exec -it fastapi-postgres psql -U postgres -d fastapi_db
```

테이블 확인

```
\dt
```

데이터 조회

```
SELECT * FROM users;
```

종료:

```
\q
```

8. 자주 쓰는 Docker 명령어

DB 시작

```
docker compose up -d
```

DB 중지

```
docker comose down
```

DB 중지 + 볼륨 삭제

이 경우 데이터도 같이 삭제 됩니다.

```
docker compose down -v
```

로그 확인

```
docker compose logs -f
```

DB 서비스 로그만 보기

```
docker compose logs -f db
```

9. OpenAPI 문서 자동 생성과 조회

FastAPI의 큰 장점 중 하나는 OpenAPI 문서를 자동 생성해준다는 점입니다.

FastAPI는 아래 정보를 바탕으로 OpenAPI 스키마를 자동으로 생성합니다.

- app= FastAPI()의 title, description, version
- 각 API 경로
- 요청 스키마 (UserCreate, UserUpdate)
- 응답 스키마 (UserResponse)
- Path Parameter (user_id)
- HTTP 메서드 (GET, POST, PUT, DELETE)

즉, 우리가 별도로 Open API YAML/JSON 파일을 손으로 작성하지 않아도 됩니다.

9-1. 자동 생성되는 문서 종류

FastAPI 실행 후 아래가 자동 제공됩니다. **Swagger UI**

```
http://127.0.0.1:8000/docs
```

특징

- 브라우저에서 직접 API 테스트 가능
- 요청 본문 샘플 자동 제공
- 응답 구조 확인 가능

ReDoc

```
http://127.0.0.1:8000/redoc
```

특징:

- 문서형으로 보기 좋음
- API 구조를 읽기 편함 **OpenAPI JSON**

```
http://127.0.0.1:8000/openapi.json
```

특징:

- 실제 OpenAPI 스키마 JSON
- Swagger UI/ReDoc도 이 JSON 기반으로 랜더링됨.

9-2. openapi.json 직접 조회하기

브라우저에 아래 주소 입력:

```
http://127.0.0.1:8000/openapi.json
```

그러면 JSON 형태의 OpenAPI 문서가 보입니다. 예를 들면 아래와 비슷한 구조가 나옵니다.

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "FastAPI + PostgreSQL CRUD Example",
    "description": "Docker Compose로 PostgreSQL을 실행하고, FastAPI + SQLAlchemy로 User CRUD를 수행하는 예제",
    "version": "1.0.0"
  },
  "paths": {
    "/users": {
      "get": {},
      "post": {}
    },
    "/users/{user_id}": {
      "get": {},
      "put": {},
      "delete": {}
    }
  }
}
```

9-3. OpenAPI 문서가 자동 생성되는 이유

우리가 main.py에서 아래와 같이 작성했기 때문입니다.

```
app = FastAPI(
    title="FastAPI + PostgreSQL CRUD Example",
    description="Docker Compose로 PostgreSQL을 실행하고, FastAPI...",
    version="1.0.0"
)
```

그리고 각 API 함수에 이런 정보가 있습니다.

```
@app.post("/users", response_model=schmas.UserResponse, tags=["Users"])
def create_user(user: schema.UserCreate, db:Session=Depends(get_db)):
    ...
```

FastAPI는 이를 해석해서:

- /users는 POST 엔드포인트
- 요청 본문은 UserCreate

- 응답은 UserResponse
- 그룹은 "Users" 라는 메타 데이터를 자동 추출합니다.

10. 전체 실행 순서 한 번에 보기

1) 프로젝트 폴더 이동

```
cd C:\Projects\fastapi-postsql-local
```

2) PostgreSQL 실행

```
docker compose up -d
```

3) 가상환경 생성

```
python -v venv .venv
```

4) 가상환경 활성화

```
venv\Scripts\activate
```

5) 패키지 설치

```
pip install -r requirements.txt
```

6) FastAPI 실행

```
uvicorn app.main:app -reload
```

7) Swagger 접속

```
http://127.0.0.1:8000/docs
```

8) OpenAPI JSON 조회

```
http://127.0.0.1:8000/openapi.json
```

9) PostgreSQL 내부확인

```
docker exec -it fastapi-postgres psql -U postgres -d fastapi_db
```

```
\dt
SELECT * from users
```

11. 핵심 포인트 정리

이 예제의 구조는 아래처럼 이해하면 됩니다.

- PostgreSQL만 Docker Compose로 구동
- FastAPI는 로컬 venv
- SQLAlchemy ORM으로 User 테이블 관리
- Pydantic으로 요청/응답 검증
- FastAPI가 OpenAPI 문서를 자동 생성
- Swagger/ReDoc/openapi.json 모두 자동 생성

12. 다음 단계로 확장할 수 있는 것

다음으로 확장할 수 있는 것은 아래 입니다.

- SQLAlchemy 2.0 쿼리 스타일 로 CRUD 개선
- .env를 더 체계적으로 관리하는 설정 클래스 도입
- Alembic으로 마이그레이션 추가
- 회원가입/로그인/JWT 인증 추가
- Ruter/Service/Repostiory 구조로 분리
- 비밀번호 해싱과 인증 미들웨어 추가